CS102 - Week 1 Notes Outline and Study Guide

Table of Contents

What is inside

• Week 1 overview and master framework: data structures, algorithms, indexed access, arrays, searching, and efficiency

• Lesson 1 - Course Orientation and What to Expect

• Lesson 2 - What Is a Data Structure?

• Lesson 3 - Indexed Data Structures

• Lesson 4 - Understanding Arrays

• Lesson 5 - Accessing Elements in Arrays

• Lesson 6 - Updating and Replacing Values

• Lesson 7 - Lists vs Arrays and Dynamic Collections

• Lesson 8 - Adding and Removing Data

• Lesson 9 - Searching Through Collections

• Lesson 10 - Introduction to Algorithms

• Lesson 11 - Linear Search

• Lesson 12 - Comparing Search Strategies

• Lesson 13 - Organizing Information Efficiently

• Lesson 14 - Week 1 Integration and Review

• Glossary, concept connections, common mistakes, review guide, cheat sheet, and exam reference

Week 1 at a Glance

Week 1 is where programming starts to move from writing code that runs to designing systems that can organize and process information well. The central theme is organization. Data structures answer, "How should this information be arranged?" Algorithms answer, "What steps should I follow to solve the problem?"

The lessons build a foundation for indexed access, arrays, collections, searching, and efficiency. You will practice thinking about data before implementation, because the structure you choose can make a solution easier to understand, easier to maintain, and faster as the amount of information grows.

• Data structures organize information so programs can access, update, search, and manage it.

• Algorithms are step-by-step procedures for solving problems.

• Indexed access lets a program retrieve information by position.

• Arrays and collections group related data instead of scattering it across separate variables.

• Sequential access checks items one at a time; direct access jumps straight to a known location.

• A strong programmer asks: what information do I have, how should it be organized, and what steps should process it?

Memory hook: Structure first. Steps second. Code third.

Made with Goodnotes

Lesson 1 - Course Orientation and What to Expect

Big Idea

CS102 shifts the focus from simply writing Python syntax to thinking about how software systems organize information. The point is not just to make code run; the point is to design code that stays understandable, maintainable, and efficient as problems get bigger.

Full Explanation

Up to this point, programming has mostly involved learning language features, writing functions, and building smaller programs. CS102 adds another layer: deciding how information should be stored, retrieved, updated, searched, and organized.

That matters because two programs can produce the same result but still differ in quality. One may be repetitive, slow, fragile, or hard to update. Another may be clean, organized, and easier to scale. Data structures and algorithms are the tools that help programmers make those design choices on purpose.

The course is less about memorizing definitions and more about building judgment. Each time you meet a new structure or algorithm, the question is not just "What is it called?" The better question is: "What problem does this help solve, and what trade-offs come with it?"

Why This Lesson Matters

Beginners often think stronger programming means learning more syntax. This lesson argues that strong programming also means learning how to design solutions before typing code.

- Think about information first.
- Think about organization second.
- Think about implementation third.

Real-World Connection

Imagine organizing thousands of photos from a horse show. If every image lands in one folder with random names, finding a specific rider becomes painful. Organizing by event, class, rider, date, or client creates a structure that makes searching easier.

That organization system is the real-world version of a data structure. The way you search through it is similar to an algorithm.

Term	Plain-English Meaning	Why It Matters
Data	Information a program can store, process, or use.	It is the raw material programs work with.
Data Structure	A method for organizing data efficiently.	It determines how easy data is to access or change.
Algorithm	A sequence of steps used to solve a problem.	It describes the process, not just the storage.
Efficiency	How much work a task requires.	Less work usually means faster, cleaner programs.
Scalability	How well a solution performs as data grows.	A good small solution should not collapse at large size.

Why Beginners Usually Miss This

- They treat the course as vocabulary memorization instead of design practice.
- They focus on code syntax and forget that organization affects performance.
- They assume there is one perfect solution instead of weighing trade-offs.

Helpful tip: When a lesson feels simple, ask what thinking habit it is trying to train.

Notes

Made with Goodnotes

Lesson 2 - What Is a Data Structure?

Big Idea

Every program works with information. A data structure is the way that information is organized so it can be accessed, updated, searched, and managed efficiently. The information itself matters, but the organization is what makes it usable.

Full Teaching Explanation

A data structure is easier to understand when you compare organized and unorganized information. A pile of random papers may contain everything you need, but finding one document takes forever. Put those same papers into labeled folders and the information becomes useful because the structure supports retrieval.

Programming works the same way. A music app stores playlists, a bank stores customer records, a navigation app stores locations, and an online store stores products and inventory. Each system needs a different structure because each system has different goals.

This is the big lesson: there is rarely one perfect way to store data. Good programmers choose a structure based on what the program needs to do most often - search, add, remove, update, preserve order, or group related information.

Unorganized Information	Organized Information	Real-Life Example	Purpose
Hard to find data	Easy to find data	Playlist	Organizes songs
Slow searching	Faster searching	Grocery receipt	Organizes purchases
Confusing structure	Predictable structure	Calendar	Organizes events
Difficult to maintain	Easier to update	Contact list	Organizes people
Increased mistakes	Reduced mistakes	Filing cabinet	Organizes documents
		Study guide	Organizes learning material

Real-World Examples

• Grocery receipt: item names, quantities, prices, discounts, taxes, and total cost are arranged predictably so questions can be answered quickly.

• Music playlist: songs are stored in a specific order so you can play next, skip back, rearrange, add, or remove songs.

• Study notes: lessons, definitions, examples, reviews, and cheat sheets are grouped so studying is not random scavenger hunting.

Programmer Thinking

Before writing code, ask: How should this information be organized? If I am building Spotify, I need playlists. If I am building a banking app, I need customer records. If I am building a school system, I need student information. The structure affects everything that comes afterward.

Made with Goodnotes

Lesson 2 - Code Pattern and Study Check

Code Example - Grouping Related Data

```
grocery_items = [
    "milk",
    "bread",
    "eggs",
    "apples"
]
print(grocery_items)
grocery_items.append("orange juice")
```

What This Code Does

The list stores several grocery items together in one organized collection. Instead of creating separate variables like `item1`, `item2`, and `item3`, the list keeps related information grouped.

The append line shows why structure matters: adding another item becomes simple because the collection already has a place for related data.

Terms and Definitions

- Data: information that can be stored and used by a program.
- Data structure: a method of organizing data so it can be used efficiently.
- Organization: arranging information in a meaningful way.
- Collection: a group of related pieces of information stored together.
- Efficiency: how much work is required to perform a task.

Common Mistakes and Helpful Tips

Mistake: Thinking data structures only exist in code.
Reality: People use organizational structures constantly.

Mistake: Memorizing definitions without understanding purpose.
Reality: The purpose matters more than the label.

Tip: For every data structure, ask what it organizes, why it is useful, what problem it solves, and when you would choose it.

Review Checkpoint

- What is a data structure?
- Why does organization matter?
- How is a playlist an example of a data structure?
- Why might two programs choose different structures for the same information?
- What role does efficiency play in data organization?

Key Takeaways

- Data structures organize information.
- Organization affects efficiency.
- Different problems require different structures.
- Real-world organizational systems are data structures too.
- Choosing the right structure makes programming easier.

Additional Notes

Made with Goodnotes

Lesson 3 - Indexed Data Structures

Big Idea

Indexed data structures make direct access possible. Instead of checking every item one at a time, an index acts like an address that tells the program exactly where a value is stored.

Full Teaching Explanation

The main problem this lesson solves is retrieval: how do we find the information we need quickly? Without an index, a program may have to use sequential access, checking items one by one until it finds the target.

An index gives the program a known location. If you know the item is in locker 327, you do not open lockers 1 through 326 first. You go directly to locker 327. That same idea applies to many programming collections.

This matters more as data grows. Searching through 5 items is not a huge deal. Searching through 5,000 or 5,000,000 items can become a serious performance problem. Indexed access reduces that work when the position is known.

Locker Hallway Mental Model

Sequential access: start at locker 1 and check each locker until you reach the right one.

Indexed access: if you already know the locker number, you walk straight to that locker.

The index works like an address. It points to the location instead of forcing a search.

Sequential Access	Indexed Access
Checks items one at a time	Jumps directly to a location
May require many checks	Usually one lookup
Gets slower as data grows	Stays fast when position is known
Like searching a pile	Like using an address

Code Example - Indexed Access

```
items = ["A", "B", "C", "D", "E", "F", "G", "H"]
print(items[7])
# Output:
# H
```

What Happened?

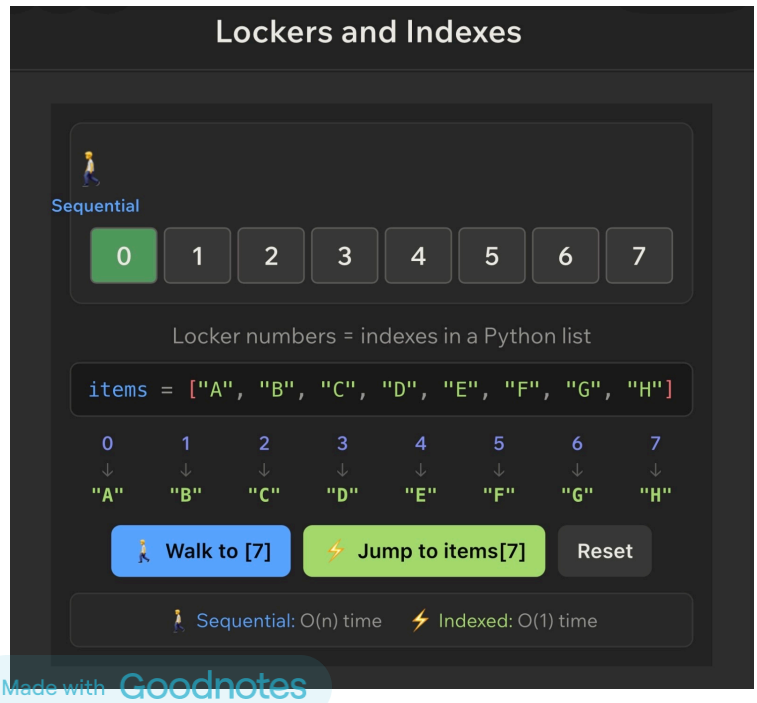
The program did not search for H. The expression items[7] told Python exactly which position to retrieve. The index served as an address.

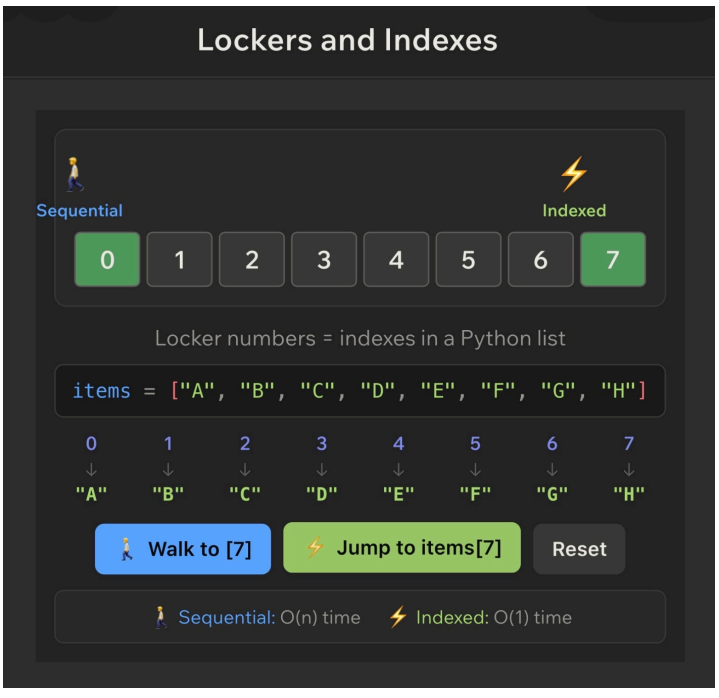
Zero-Based Indexing

Python positions start at 0, not 1. This is one of the most common beginner mistakes.

Human counting: 1 2 3 4 5
Python indexing: 0 1 2 3 4

Example: in colors = ["red", "blue", "green"], colors[0] returns red and colors[2] returns green.





Made with Goodnotes

Lesson 3 - Study Check

Terms and Definitions

• Index: a numerical position within a collection, such as the 3 in items[3].

• Indexed access: using an index to retrieve information directly.

• Sequential access: checking items one at a time until the desired item is found.

• Position: the location of an item inside a collection.

• Zero-based indexing: a numbering system where positions begin at 0 instead of 1.

Common Mistakes and Helpful Tips

Mistake: assuming the first position is 1.
Reality: the first position is 0.

Mistake: confusing the value with the index. colors[1] does not mean "blue is 1"; it means retrieve the value stored at position 1.

Mistake: accessing an index that does not exist, like colors[5] in a three-item collection.

Tip: draw indexes above values whenever you feel unsure.

Index	0	1	2	3
Value	A	B	C	D

Review Checkpoint

• What is an index?

• How does indexed access differ from sequential access?

• Why are indexes useful?

• What is zero-based indexing?

• What value would print from items = ["A", "B", "C", "D"]; print(items[2])?

• Why does direct access become more valuable as collections grow?

Key Takeaways

• Indexes act like addresses.

• Indexed access allows direct retrieval.

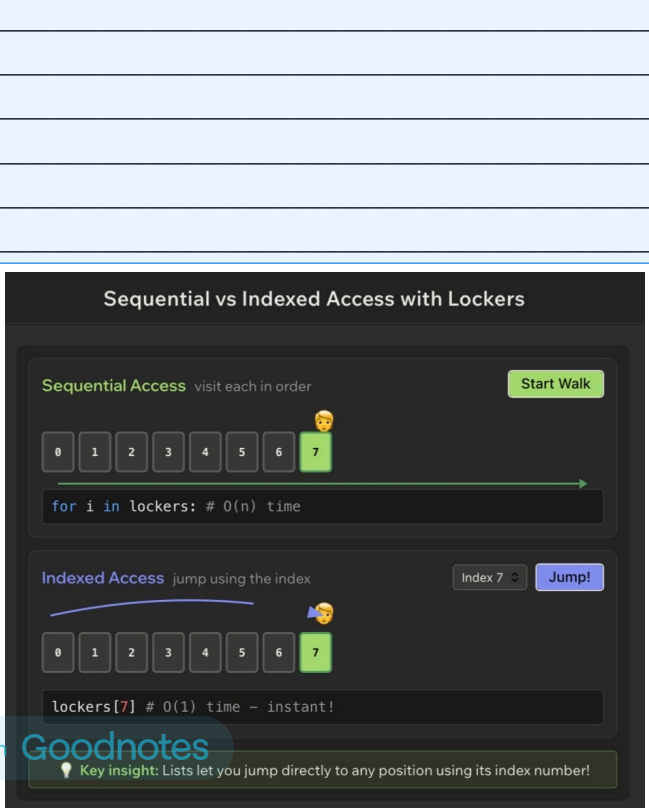
• Sequential access examines items one at a time.

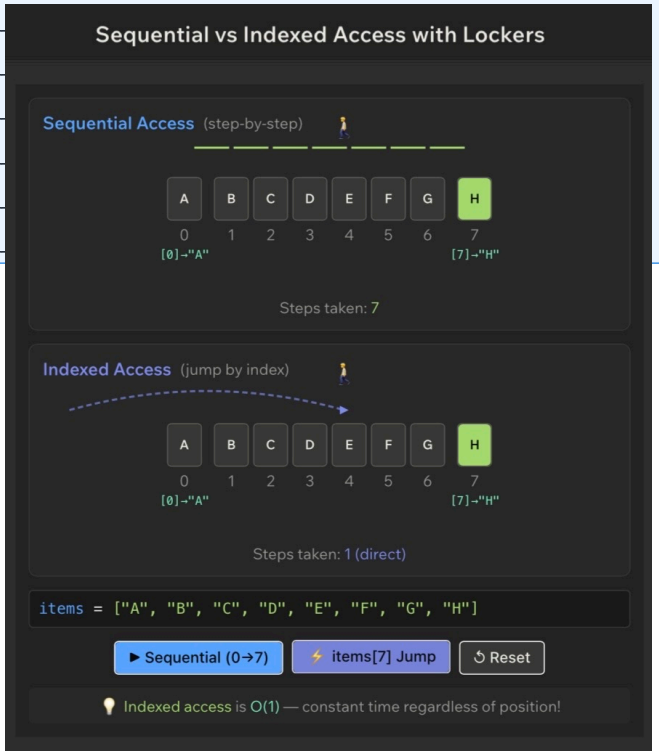
• Python uses zero-based indexing.

• Direct access becomes more valuable as collections grow.

• Understanding indexes is foundational for arrays and lists.

Notes





Made with Goodnotes

Lesson 4 - Understanding Arrays

Big Idea

Arrays store information in an organized sequence of positions. Each position has an index, which means a program can retrieve a value directly when it knows where that value lives.

Full Teaching Explanation

Lesson 4 connects two earlier ideas: organization and indexing. An array is not just a loose pile of values. It is a structure where values live in predictable positions.

A movie theater is the perfect mental model. If your ticket says Row B, Seat 3, you do not inspect every seat until you find yours. The row and seat number tell you exactly where to go. In arrays, the index plays that same role.

Arrays are especially useful when order matters, positions matter, fast access is needed, or similar kinds of data are stored together. Student grades, product prices, daily temperatures, survey responses, game scores, and roster names are all natural array-style examples.

Movie Theater Analogy

Theater seat number = array index

Person sitting there = value

Theater layout = array structure

Finding Seat 4 = accessing index 4

Why Arrays Matter

- They preserve order.
- They store related values together.
- They support direct access by index.
- They appear everywhere: e-commerce, education, gaming, and weather systems.

Index	0	1	2	3	4
Value	A	B	C	D	E

Code Example - Accessing an Array Position

```
letters = ["A", "B", "C", "D", "E"]
print(letters[3])
# Output:
# D
```

Why This Works

The array stores information in specific positions. D is stored at index 3, so Python retrieves the value at position 3. It does not search for D; it goes directly to the location.

Made with Goodnotes

Lesson 4 - Arrays in Practice

Code Example - Student Scores

```
scores = [85, 92, 78, 100, 88]
print(scores[1])
# Output:
# 92
```

Programmer Thinking

When I look at an array, I do not just think, "This is a list of values." I think, "This is a collection of positions. Each position contains a value, and the index tells me where that value lives."

Terms and Definitions

- Array: a collection of values stored in an ordered sequence.
- Element: an individual value stored inside an array.
- Index: the position of an element.
- Ordered collection: a collection where positions matter and order is preserved.
- Direct access: retrieving a value immediately by using its index.

Common Mistakes and Helpful Tips

Mistake: thinking arrays store information randomly.
Reality: arrays store values in a specific order.

Mistake: forgetting indexes begin at zero.
Reality: the first element is array[0].

Mistake: confusing element number with index number. The first element is at index 0, not index 1.

Tip: draw the index row above the value row.

Review Checkpoint

- What is an array?
- How is a movie theater similar to an array?
- What is an element?
- What is an index?
- Why are arrays considered ordered collections?
- What value is returned by items = ["A", "B", "C", "D"]; print(items[3])?
- Why is direct access useful?

Key Takeaways

- Arrays store information in ordered positions.
- Each position has an index.
- Indexes allow direct access.
- Arrays preserve order.
- Elements are individual values stored in the array.
- Arrays are foundational data structures used throughout programming.

Notes

Made with Goodnotes

Lesson 5 - Accessing Elements in Arrays

Big Idea

Once information is stored in an array, programmers need a way to retrieve specific values. Accessing elements means using an index to get the data stored at a particular position.

Full Teaching Explanation

An array is useful because the information inside it can be accessed. If someone gives you hotel room 214, you know exactly where to go. The room number works like an address. In an array, the index works the same way.

The access pattern is simple: identify the position needed, use the index, then retrieve the value. This is fast and predictable as long as the position exists.

That last part matters. If you ask for an index outside the valid range, Python cannot retrieve a value because there is no value stored there. That creates an IndexError.

Index	0	1	2	3
Value	red	blue	green	yellow

Array Access Pattern

Step 1: Identify the position needed.

Step 2: Use the index.

Step 3: Retrieve the value.

Example: colors[2] retrieves green.

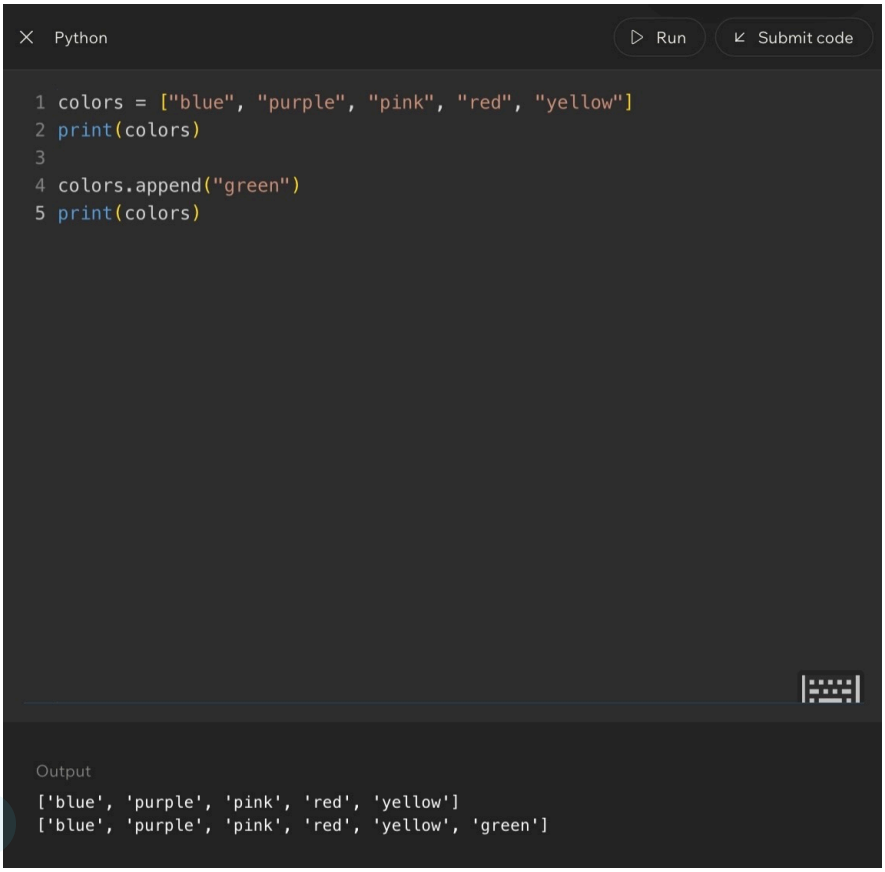
Code Example - Basic Access

```
colors = ["red", "blue", "green", "yellow"]
print(colors[2])
# Output:
# green
```

Code Example - First and Last Elements

```
animals = ["dog", "cat", "bird", "fish"]
print(animals[0])
print(animals[3])
# Output:
# dog
# fish
```

Code Example:



Made with Goodnotes

Lesson 5 - Adding, Inserting, Extending, and Index Errors

Operation	What It Does	Mental Model
append(value)	Adds one element to the end.	Put this new item at the next available position.
insert(index, value)	Places a value at a specific position.	Put it here, then shift later items right.
extend(other)	Adds every element from another collection.	Combine these collections into one.

append() Example

```
colors = ["red", "blue"]
colors.append("green")
print(colors)
# ['red', 'blue', 'green']
```

insert() Example

```
nums = [10, 30, 40]
nums.insert(1, 20)
print(nums)
# [10, 20, 30, 40]
```

extend() Example

```
nums = [1, 2, 3]
more_nums = [4, 5, 6]
nums.extend(more_nums)
print(nums)
# [1, 2, 3, 4, 5, 6]
```

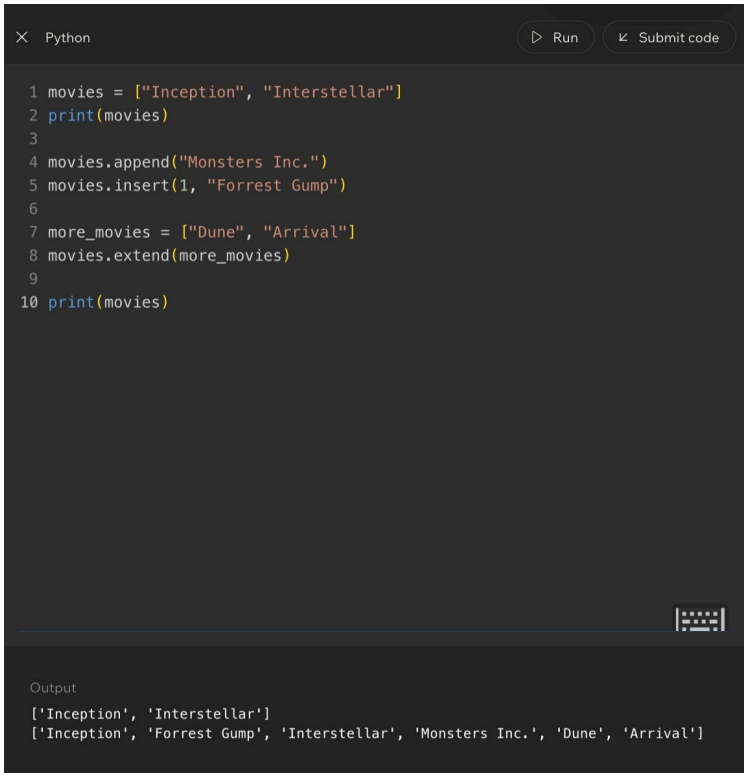
Invalid Indexes and Errors

A valid index must exist inside the collection. If colors has indexes 0, 1, and 2, then colors[3], colors[4], and colors[5] are invalid.

Example: colors = ["red", "blue", "green"]; print(colors[5]) causes IndexError because position 5 does not exist.

Code Example:





Made with Goodnotes

Lesson 5 - Study Check

Terms and Definitions

- Access: retrieving information from a collection.
- append(): adds an element to the end of a collection.
- insert(): places an element at a specific position.
- extend(): combines elements from another collection.
- IndexError: an error caused by requesting an invalid position.
- Valid index: an index that exists within the collection.

Common Mistakes and Helpful Tips

Mistake: assuming indexes start at 1.
Reality: indexes start at 0.

Mistake: using an index larger than the collection size.
Reality: only existing positions can be accessed.

Mistake: confusing append() and insert(). append() adds to the end; insert() places a value at a specific location.

Tip: when debugging index problems, write indexes above the values.

Review Checkpoint

- What does an index represent?
- How does append() work?
- How does insert() differ from append()?
- What does extend() do?
- What causes an IndexError?
- What prints after nums = [10, 20]; nums.append(30); print(nums)?
- What prints after nums = [10, 30]; nums.insert(1, 20); print(nums)?

Key Takeaways

- Arrays use indexes for direct access.
- append() adds to the end.
- insert() adds at a specific position and shifts later values.
- extend() combines collections.
- Invalid positions cause IndexError.
- Understanding indexes is critical for working with collections.

Notes

Made with Goodnotes

Lesson 6 - Updating and Replacing Values in Arrays

Big Idea

Arrays are not just for storing and reading data. Real programs constantly update information: grades change, inventory counts drop, user profiles get edited, and seat assignments become occupied. Updating means replacing the value stored at an existing index.

Full Teaching Explanation

The key idea is that the location stays the same, but the value changes. In a gradebook, Student 3 may stay in the same row, but their score can change from 78 to 82 after a correction. Arrays work the same way: the index remains the address, and the value stored there is replaced.

This is different from append() or insert(). append() adds something new to the end. insert() places something new at a specific position and shifts other values. Updating does not create a new position. It overwrites the value already stored at that index.

This matters because software would become outdated quickly if it could only display old values. Banking balances, product inventory, grades, social media profiles, and game stats all depend on updates.

Gradebook Mental Model

Before update: Student 3 has 78.

After correction: Student 3 has 82.

The student did not move. Only the score stored in that position changed.

	Index 0	Index 1	Index 2	Index 3
Before	85	90	78	88
After	85	90	82	88

Code Example - Updating a Value

```
scores = [85, 90, 78, 88]
scores[2] = 82
print(scores)
# Output:
# [85, 90, 82, 88]
```

What Happened?

scores[2] = 82 means: go to index 2 and replace the current value with 82. Python does not create a new position; it overwrites the existing value.

Code Example:

Python ▷ Run ↙ Submit code

```
1 movies = ["Encanto", "Lilo & Stitch", "Coco", "Frozen", "Monster's Inc"]
2 print(movies)
3
4 movies[0] = "Mighty Ducks"
5 movies[len(movies) // 2] = "Hocus Pocus"
6 movies[4] = "Cinderella"
7
8 print(movies)
```

Output

```
['Encanto', 'Lilo & Stitch', 'Coco', 'Frozen', "Monster's Inc"]
['Mighty Ducks', 'Lilo & Stitch', 'Hocus Pocus', 'Frozen', 'Cinderella']
```

Made with Goodnotes

Lesson 6 - First, Middle, Last, and Update Thinking

Code Example - Updating a Name

```
students = ["Sophia", "CJ", "Caleb"]
students[1] = "Christopher"
print(students)
# Output:
# ['Sophia', 'Christopher', 'Caleb']
```

Common Positions

First element: always index 0.

Last element: depends on the length. In a 4-item array, the last index is 3.

Middle element: for [10, 20, 30, 40, 50], the middle value is 30 at index 2.

Position	Index	Value
First	0	10
Middle	2	30
Last	4	50

Programmer Thinking

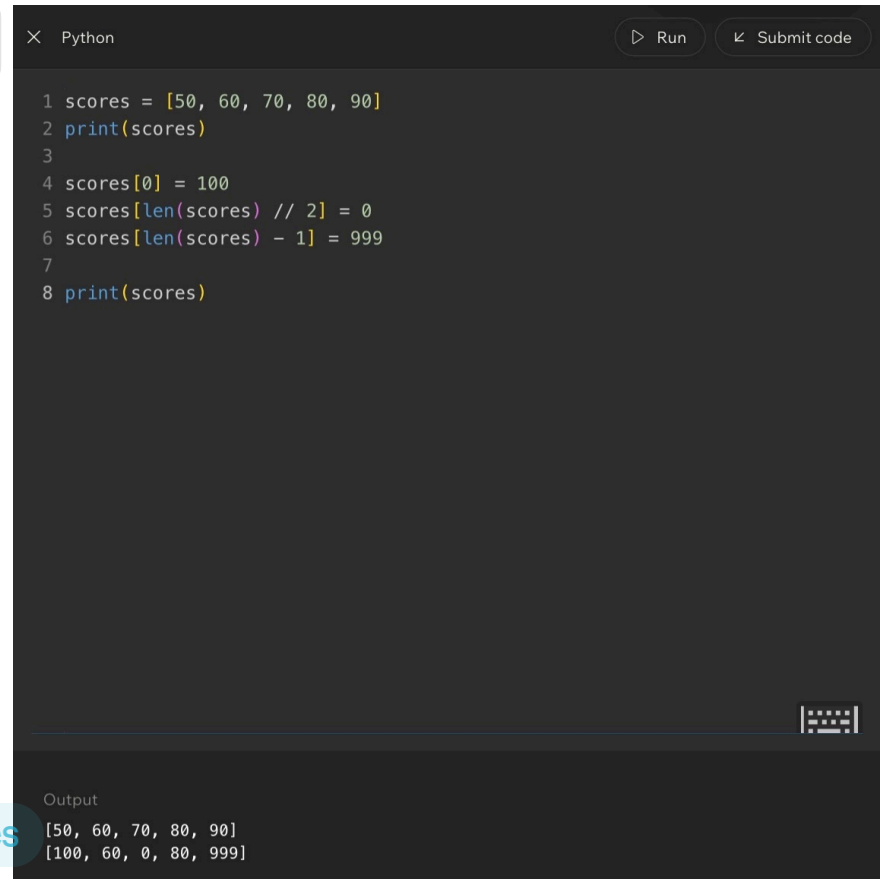
When I look at an array, I ask: am I trying to add something, replace something, or remove something? If I am replacing information, I do not need append() or insert(). I already know where the information belongs. I simply update the value at that location.

Code Example - Updating Inventory

```
inventory = [12, 8, 15]
inventory[1] = 6
print(inventory)
# Output:
# [12, 6, 15]
```

System	What Changes
Banking	Account balances change after deposits and withdrawals.
E-commerce	Inventory counts change when products are sold.
Education	Grades change when assignments are entered.
Social Media	Profile information changes when users edit accounts.

Code Example:



Made with Goodnotes

Lesson 6 - Study Check

Terms and Definitions

- Update: changing an existing value.
- Replace: substituting one value for another.
- Assignment: storing a value in a variable or location, such as scores[2] = 82.
- Existing element: a value already stored in a collection.
- Position: the location of a value within an array.

Common Mistakes and Helpful Tips

Mistake: using append() when you mean to replace a value.
Reality: append() adds a new element; updating changes an existing one.

Mistake: forgetting which index needs to be updated.
Reality: draw the indexes first.

Mistake: confusing value with position.
Reality: the index identifies the location; the value is the information stored there.

Tip: before updating, ask: which element changes, what index contains it, and what should the new value be?

Review Checkpoint

- What does it mean to update an array?
- How is updating different from appending?
- What happens when you assign a new value to an existing index?
- Where is the first element located?
- How can you identify the middle element?
- What prints after nums = [10, 20, 30]; nums[1] = 99; print(nums)?
- Why are updates important in real-world software?

Key Takeaways

- Arrays can store changing information.
- Updating replaces an existing value.
- The index stays the same while the stored value changes.
- First, middle, and last positions are common access patterns.
- Updating data is one of the most common operations in software development.

Notes

_____

_____

_____

_____

_____

Made with Goodnotes

Lesson 7 - Dynamic Collections and Why Arrays Aren't Always Enough

Big Idea

Arrays are powerful because they store values in predictable positions, but real-world applications rarely stay the same size forever. Dynamic collections provide flexibility when data needs to grow, shrink, or reorganize over time.

Full Teaching Explanation

Traditional arrays were designed with fixed sizes. Once created, they could not easily grow or shrink. If more space was needed, programmers often had to create a new array and copy the existing information into it.

Modern languages give us flexible structures that can expand automatically, but the underlying idea still matters. Every data structure makes tradeoffs. Some prioritize speed. Some prioritize flexibility. Some prioritize memory efficiency.

The structure that works well for ten items may become a poor choice for ten million items. This lesson trains the habit of asking whether a collection will stay roughly the same size or keep changing.

Bookshelf Mental Model

A shelf that holds exactly 20 books works fine until you buy 10 more books.

Option 1: buy a larger shelf and move everything.

Option 2: add another shelf somewhere else.

Neither is perfect; both involve work. Data structures face similar growth decisions.

Fixed Collection	Dynamic Collection
Capacity is limited	Can grow or shrink
May require rebuilding	Adapts as data changes
Predictable size	More flexible
Good when size is known	Good when size changes

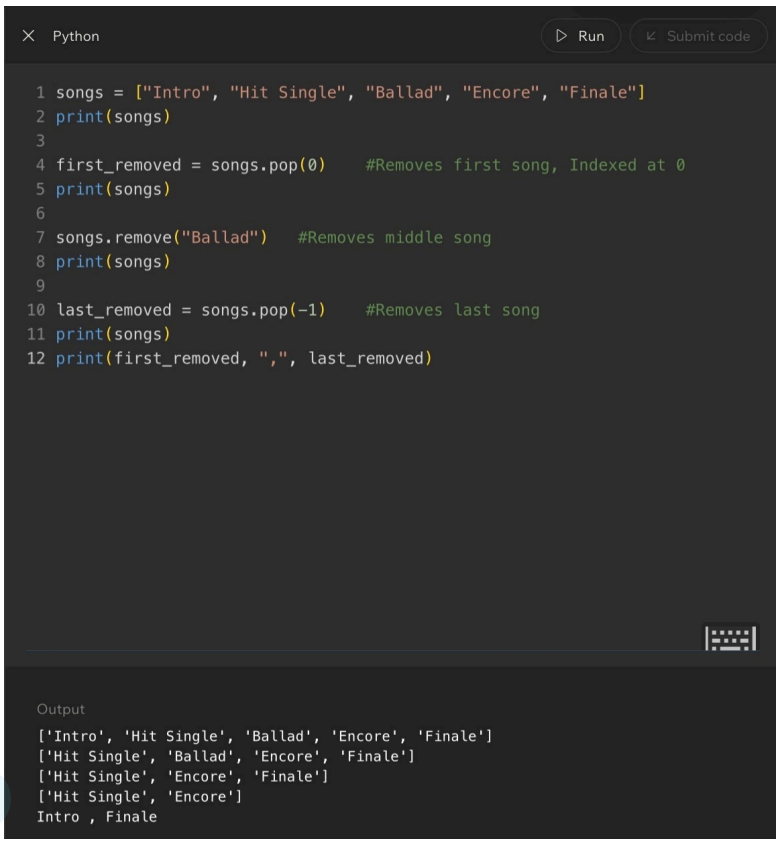
Code Example - Growing a Collection

```
colors = ["red", "blue", "green"]
colors.append("yellow")
colors.append("purple")
print(colors)
# Output:
# ['red', 'blue', 'green', 'yellow', 'purple']
```

Why This Matters

Spotify playlists, Amazon product catalogs, friend lists, and customer databases all grow over time. If every new item required rebuilding the whole collection, performance would suffer dramatically.

Code Example:



Made with Goodnotes

Lesson 7 - Study Check

Programmer Thinking

When designing a system, ask: will this collection stay roughly the same size, or will it grow constantly? If growth is expected, flexibility becomes an important design consideration.

Terms and Definitions

- Dynamic collection: a collection capable of growing and shrinking as needed.
- Fixed size: a structure whose capacity cannot easily change.
- Capacity: the amount of information a structure can hold.
- Resizing: expanding or shrinking a collection.
- Tradeoff: improving one characteristic may require sacrificing another.

Common Mistakes and Helpful Tips

Mistake: assuming bigger is always better.
Reality: different structures solve different problems.

Mistake: thinking all collections grow the same way.
Reality: growth strategies vary between structures.

Tip: whenever learning a new structure, ask how data is stored, how data is retrieved, how it handles growth, and what strengths/weaknesses it has.

Review Checkpoint

- Why might a fixed-size structure become problematic?
- What is a dynamic collection?
- What does capacity mean?
- Why are tradeoffs important?
- How do growing collections differ from fixed collections?

Key Takeaways

- Arrays are useful but not perfect.
- Some structures must adapt to growth.
- Dynamic collections provide flexibility.
- Every data structure involves tradeoffs.
- Choosing the right structure depends on the problem.

Notes

Made with Goodnotes

Lesson 8 - Adding and Removing Data Efficiently

Big Idea

Adding and removing data changes a collection's organization. Adding to the end is often simple, but inserting into the middle or removing from the middle can force other elements to shift positions.

Full Teaching Explanation

Imagine a line at a movie theater: A B C D E. If someone needs to be inserted between B and C, the line becomes A B X C D E. Everyone after B shifts one position. Indexed collections often behave the same way.

Removing information creates a similar challenge because the collection must stay organized after the removal. The empty space needs to be closed, which usually means later values shift left.

This movement may feel small with five items, but it becomes much more expensive in collections with millions of elements. Understanding what moves helps explain why some operations are faster than others.

Operation	What Happens	Efficiency Concern
Add to end	New item goes after the last element.	Usually simple.
Insert in middle	Later elements shift right.	Can require moving many elements.
Remove from middle	Later elements shift left.	Indexes may change after removal.

Before Insert	Insert X at Index 2
Index: 0 1 2 3 Value: A B C D	Index: 0 1 2 3 4 Value: A B X C D C and D moved right.

Code Example - Insert

```
letters = ["A", "B", "C", "D"]
letters.insert(2, "X")
print(letters)
# Output:
# ['A', 'B', 'X', 'C', 'D']
```

Made with Goodnotes

Lesson 8 - Remove, Shift, and Study Check

Code Example - Remove

```
letters = ["A", "B", "C", "D"]
letters.remove("C")
print(letters)
# Output:
# ['A', 'B', 'D']
```

Programmer Thinking

When I insert something into the middle of a collection, I immediately ask: what else has to move? That question helps explain why some operations are more expensive than others.

Terms and Definitions

- Insert: adding a value at a specific position.
- Remove: deleting a value from a collection.
- Shift: moving elements to new positions.
- Modification: any change made to a collection.
- Reorganization: adjusting positions after changes occur.

Common Mistakes and Helpful Tips

Mistake: assuming insert only affects one position.
Reality: many elements may need to move.

Mistake: forgetting indexes change after removals.
Reality: positions often shift after data is removed.

Tip: draw the collection before and after modifications to visualize movement.

Review Checkpoint

- What happens when an element is inserted in the middle?
- Why might inserts require shifting?
- What happens when elements are removed?
- Why do indexes change after removals?
- How does collection size affect modification costs?

Key Takeaways

- Adding and removing data affects organization.
- Inserts often cause shifting.
- Removals often change indexes.
- Collection size influences efficiency.
- Understanding movement helps explain performance differences.

Notes

Made with Goodnotes

Lesson 9 - Searching Through Collections

Big Idea

Searching is how programmers locate specific information inside a collection. Once data is stored, the next question is: how do we find the one item we need quickly and accurately?

Full Explanation

Organization and searching work together. A messy pile of information is harder to search than a structured collection. Search algorithms give programs a systematic process for checking items, identifying matches, and deciding what to do when the target is found or missing.

In this lesson, the important shift is from “I have data” to “I need a repeatable way to find something inside that data.” The structure affects how much work the search requires.

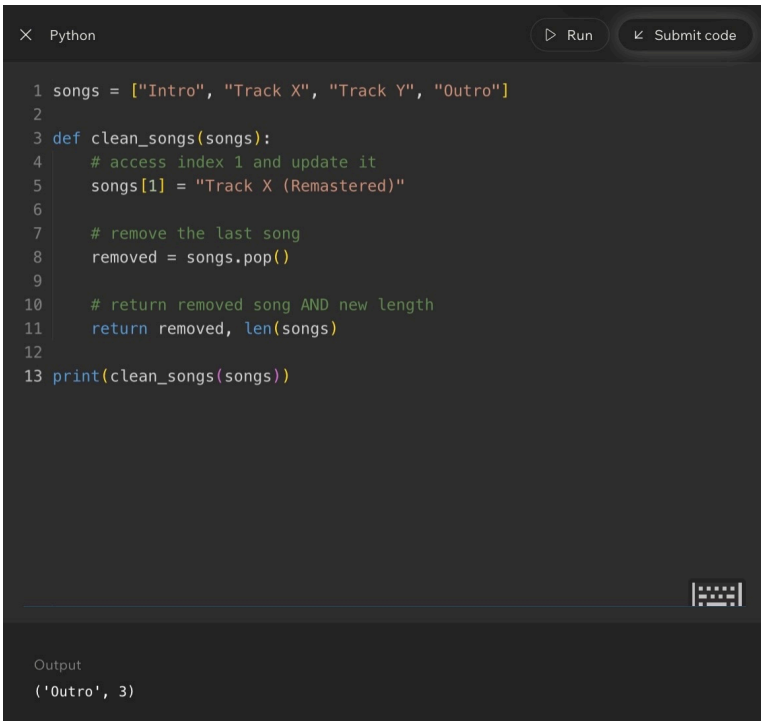
Real-World Examples

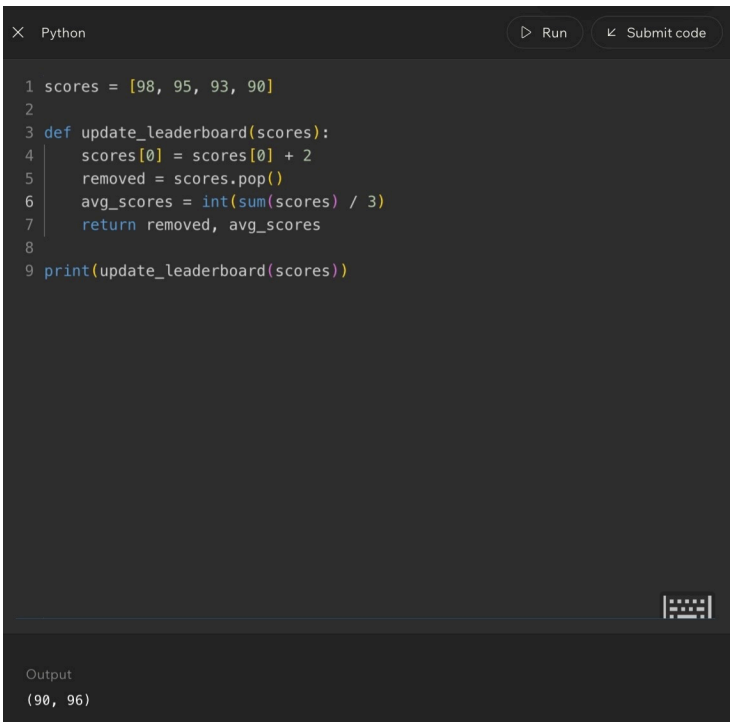
- Class roster: If the roster is random, you may read one name at a time. If it is alphabetized, you can search more efficiently.
- Playlist: Looking through 5,000 songs manually is slow. Typing the song title lets the app locate it much faster.
- Photography files: Finding one rider image is easier when photos are organized by event, class, and rider instead of dumped into one folder.

Search Pattern

Step	Question
1. Start	Choose where the search begins.
2. Check item	Compare the current item to the target.
3. Match?	If yes, stop. If no, continue.
4. Finish	Return success or report that the item was not found.

Code Example:





Made with Goodnotes

Lesson 9 - Searching Through Collections

Code Example - Simple Search

```
colors = ["red", "blue", "green", "yellow"]
target = "green"

for color in colors:
    if color == target:
        print("Found!")
```

Plain English: Look at each color. If the current color equals the target, print that it was found.

Terms and Definitions

Search: The process of locating specific information.

Search Algorithm: A set of steps used to find information.

Target: The value being searched for, such as `target = "green"`.

Match: When the current item equals the target.

Collection: A group of related data stored together.

Common Mistakes and Helpful Tips

- Mistake: Thinking searching only exists in programming. Reality: people search constantly in everyday life.
- Mistake: Ignoring how the information is organized. Reality: organization often determines search efficiency.
- Tip: Ask what you are searching for, how the data is organized, how many items exist, and what happens if the item is not found.

Key Takeaways

- Searching locates information inside collections.
- Search algorithms provide systematic steps.
- Organization affects search efficiency.
- Data structures and algorithms work together.

Notes

Made with Goodnotes

Lesson 10 - Introduction to Algorithms

Big Idea

Algorithms are step-by-step procedures used to solve problems. Data structures organize information; algorithms process that information.

Full Explanation

An algorithm does not have to be code. A recipe, GPS directions, a morning routine, or instructions for assembling furniture are all algorithms because they are ordered, repeatable steps designed to reach a goal.

In computer science, algorithms tell software what actions to perform: search, sort, calculate, process, recommend, or decide. Good algorithms reduce confusion and unnecessary work.

Everyday Algorithm Examples

Example	Algorithm Thinking
Recipe	Follow ordered steps to create food.
GPS directions	Take each turn in sequence to reach a destination.
Morning routine	Repeat a familiar process each day.
Search feature	Follow steps to locate a target value.

Problem -> Algorithm -> Solution

Problem: Need to find a book.

Algorithm: Go to shelf -> check title -> match? -> continue if necessary.

Solution: Book found, or search ends with no match.

Code Example:

Python — Run — Submit code

```
guests = ["Ana", "Ben", "Chris", "Dana", "Eli"]

def remove_middle_guest(guests):
    middle_guest = len(guests) // 2
    removed_guest = guests.pop(middle_guest)
    return guests

print(remove_middle_guest(guests))
```

Output

```
['Ana', 'Ben', 'Dana', 'Eli']
```

Python — Run — Submit code

```
guests = ["Ana", "Ben", "Chris", "Dana", "Eli", "Beth"]

def remove_middle_guest(guests):
    if len(guests) % 2 == 0:
        print("Guest list unchanged. List is empty or even numbered.")
        print(guests)
    else:
        middle_guest = len(guests) // 2
        removed_guest = guests.pop(middle_guest)
        return guests

print(remove_middle_guest(guests))
```

Output

```
Guest list unchanged. List is empty or even numbered.
['Ana', 'Ben', 'Chris', 'Dana', 'Eli', 'Beth']
None
```

Made with Goodnotes

Lesson 10 - Introduction to Algorithms

Code Example - Repeating a Process

```
numbers = [4, 8, 15, 16, 23, 42]

for number in numbers:
    print(number)
```

What this algorithm does: Starts with the first number, prints it, moves to the next number, and repeats until the collection is finished.

Think Like a Programmer

When you hear “algorithm,” do not jump straight to complicated math. Think: a repeatable set of instructions. Explain it in plain English first, then translate it into code.

Terms and Definitions

Algorithm: A sequence of steps used to solve a problem.

Procedure: A set of actions performed in order.

Step: One individual action inside an algorithm.

Process: A group of actions working toward a goal.

Logic: The reasoning behind decisions and actions.

Common Mistakes and Helpful Tips

- Mistake: Thinking algorithms are always code. Reality: algorithms exist before code is written.
- Mistake: Assuming algorithms must be complicated. Reality: even simple instructions can be algorithms.
- Tip: If you can explain the steps to another person, you are more likely to understand the code version.

Key Takeaways

- Algorithms solve problems using ordered steps.
- Algorithms exist inside and outside programming.
- Data structures organize information; algorithms process it.
- A clear plain-English algorithm makes the code easier to write.

Notes

Made with Goodnotes

Lesson 11 - Linear Search

Big Idea

Linear Search is one of the simplest search algorithms. It checks items one at a time until it finds the target or reaches the end of the collection.

Full Explanation

Linear Search combines collections and algorithms. The collection holds the data. The algorithm tells the program how to move through that data. It begins at the first element, compares it to the target, and continues forward until there is a match or nothing is left to check.

It is simple and reliable because it works even when the collection is not sorted. The tradeoff is that large collections may require many comparisons.

Real-World Examples

- Looking for a name on an unsorted roster: read one name at a time until Sophia appears.
- Looking through mail: check each envelope until the electric bill appears.
- Searching a drawer: check each shirt until you find the one you wanted.

Linear Search Process

Collection	Target	Checks
A B C D E F G	F	A -> B -> C -> D -> E -> F -> Found
A B C D E F G	G	A -> B -> C -> D -> E -> F -> G -> Found
A B C D E F G	Z	Check every item -> Not found

Made with Goodnotes

Lesson 11 - Linear Search

Code Example - Basic Linear Search

```
colors = ["red", "blue", "green", "yellow"]
target = "green"

for color in colors:
    if color == target:
        print("Found!")
```

Plain English: Check red, then blue, then green. Green matches the target, so the search succeeds.

Code Example - Search Failure

```
numbers = [10, 20, 30, 40, 50]
target = 100
found = False

for number in numbers:
    if number == target:
        found = True

print(found)
```

Output: False. The algorithm checked every element and never found the target.

Worst-Case Scenario

The worst case happens when the target is last or does not exist. In that situation, Linear Search may have to check every element in the collection.

Made with Goodnotes

Lesson 11 - Linear Search

Out-Loud Programmer Thinking

Linear Search means: start at the beginning and check everything until you find what you need. It is not fancy, and it is not always fast, but it is reliable and works on unsorted data.

Terms and Definitions

Linear Search: A search algorithm that checks elements one at a time.
Target: The value being searched for.
Match: When the current element equals the target.
Search Failure: When the target does not exist in the collection.
Sequential Access: Examining items one after another.

Common Mistakes and Helpful Tips

- Mistake: Assuming the search always finds something. Reality: the target may not exist.
- Mistake: Forgetting to stop after finding a match. Reality: many searches can end immediately after success.
- Mistake: Assuming Linear Search is always fast. Reality: large collections may require many comparisons.
- Tip: Physically trace the search path on paper. Watching each comparison makes the algorithm easier to understand.

Review Checkpoint

1. What is Linear Search?
2. How does it work?
3. What happens if the target is not found?
4. Why is Linear Search considered sequential?
5. What is the worst-case scenario for Linear Search?

Key Takeaways

- Linear Search examines items one at a time.
- It starts at the beginning of the collection.
- It stops when the target is found or the collection ends.
- It works on unsorted collections.
- Simplicity is one of its biggest strengths.

Notes

Made with Goodnotes

Lesson 12 - Comparing Search Strategies

Big Idea

Not all search methods require the same amount of work. Two algorithms can both produce the correct answer, but one may take dramatically longer than the other.

Full Explanation

This lesson separates two ideas that beginners often combine: correctness and efficiency. A correct algorithm gives the right answer. An efficient algorithm gives the right answer while doing less unnecessary work.

As collections grow, the difference matters more. A search strategy that feels fine with 10 items may become frustratingly slow with 10 million items. Strong programmers compare solutions instead of stopping at the first one that works.

Real-World Example - Dictionary Search

- Student A starts at page 1 and reads forward until the word appears.
- Student B opens near the middle, decides whether the word comes before or after that point, and narrows the search.
- Both can be correct. Student B usually does much less work.

Correctness vs. Efficiency

Question	What It Tells You
Does it work?	Correctness: the solution produces the right answer.
How much work does it require?	Efficiency: the solution reaches the answer with less wasted effort.
What happens when the data grows?	Scalability: the solution still performs well with larger collections.

Code Example:

Python

Run Submit code

```python
garage = [
    [  # floor 0
        ["empty", "empty", "empty"],  # row 0
        ["empty", "empty", "empty"]   # row 1
    ],
    [  # floor 1
        ["empty", "empty", "empty"],  # row 0
        ["empty", "empty", "empty"]   # row 1
    ]
]

print("Garage Floor 1:")
print("Row 0:", garage[1][0])
print("Row 1:", garage[1][1])

garage[1][0][2] = "ABC123"
print("Garage Floor 1 (Updated):")
print("Row 0:", garage[1][0])
print("Row 1:", garage[1][1])

print("Garage Floor 0:")
print("Row 0:", garage[0][0])
print("Row 1:", garage[0][1])

garage[0][1][0] = "XYZ999"
print("Garage Floor 0 (Updated):")
print("Row 0:", garage[0][0])
print("Row 1:", garage[0][1])
```

Output

```
Garage Floor 1:
Row 0: ['empty', 'empty', 'empty']
Row 1: ['empty', 'empty', 'empty']
Garage Floor 1 (Updated):
Row 0: ['empty', 'empty', 'ABC123']
Row 1: ['empty', 'empty', 'empty']
Garage Floor 0:
Row 0: ['empty', 'empty', 'empty']
Row 1: ['empty', 'empty', 'empty']
Garage Floor 0 (Updated):
Row 0: ['empty', 'empty', 'empty']
Row 1: ['XYZ999', 'empty', 'empty']
```

Made with Goodnotes

Lesson 12 - Comparing Search Strategies

Code Example - Correct, But Still Sequential

```
numbers = [1, 2, 3, 4, 5]
target = 5

for number in numbers:
    if number == target:
        print("Found")
```

Plain English: This works, but it still checks items one at a time. With a tiny list, that is fine. With a huge list, the amount of work matters.

Why This Matters as Data Grows

Collection Size	What Happens
10 items	Linear search usually feels fast.
1,000 items	The extra work becomes noticeable.
1,000,000 items	A poor strategy may require enormous work.

Terms and Definitions

Efficiency: The amount of work required to solve a problem.
Correctness: Producing the right answer.
Comparison: Evaluating different approaches.
Search Strategy: The method used to locate information.
Scalability: How well performance holds up as data grows.

Common Mistakes and Helpful Tips

- Mistake: Thinking the first working solution is automatically the best solution.
- Mistake: Ignoring efficiency because the small example runs quickly.
- Tip: Ask whether it works, whether it can be improved, and what happens if the data becomes much larger.

Key Takeaways

- Correctness and efficiency are different concepts.
- Two algorithms may produce the same answer.
- Some algorithms require more work than others.
- Efficiency becomes more important as data grows.
- Comparing solutions is a key software engineering skill.

Notes

Made with Goodnotes

Code Example:

Python

Run | Submit code

```
garage = [
    [  # floor 0
        ["empty", "empty", "empty"],  # row 0
        ["empty", "empty", "empty"]   # row 1
    ],
    [  # floor 1
        ["empty", "empty", "empty"],  # row 0
        ["empty", "empty", "empty"]   # row 1
    ]
]

print("Garage Floor 1:")
print("Row 0:", garage[1][0])
print("Row 1:", garage[1][1])

garage[1][0][2] = "ABC123"
plate = garage[1][0][2]
print("Garage Floor 1 (Updated):")
print("Row 0:", garage[1][0])
print("Row 1:", garage[1][1])
print("Plate:", plate)

print("Garage Floor 0:")
print("Row 0:", garage[0][0])
print("Row 1:", garage[0][1])

garage[0][1][0] = "XYZ999"
print("Garage Floor 0 (Updated):")
print("Row 0:", garage[0][0])
print("Row 1:", garage[0][1])
```

Output

```
Garage Floor 1:
Row 0: ['empty', 'empty', 'empty']
Row 1: ['empty', 'empty', 'empty']
Garage Floor 1 (Updated):
Row 0: ['empty', 'empty', 'ABC123']
Row 1: ['empty', 'empty', 'empty']
Plate: ABC123
Garage Floor 0:
Row 0: ['empty', 'empty', 'empty']
Row 1: ['empty', 'empty', 'empty']
Garage Floor 0 (Updated):
Row 0: ['empty', 'empty', 'empty']
Row 1: ['XYZ999', 'empty', 'empty']
```

Python

Run | Submit code

```
garage = [
    [  # floor 0
        ["empty", "empty", "empty"],  # row 0
        ["empty", "empty", "empty"]   # row 1
    ],
    [  # floor 1
        ["empty", "empty", "empty"],  # row 0
        ["empty", "empty", "empty"]   # row 1
    ]
]

garage[0][0][1] = "AAA111"
garage[0][1][2] = "BBB222"
garage[1][1][0] = "CCC333"
garage[0][1][2] = "empty"

print("Garage Floor 0:")
print("Row 0:", garage[0][0])
print("Row 1:", garage[0][1])

print("Garage Floor 1:")
print("Row 0:", garage[1][0])
print("Row 1:", garage[1][1])
```

Output

```
Garage Floor 0:
Row 0: ['empty', 'AAA111', 'empty']
Row 1: ['empty', 'empty', 'empty']
Garage Floor 1:
Row 0: ['empty', 'empty', 'empty']
Row 1: ['CCC333', 'empty', 'empty']
```

Made with Goodnotes

Lesson 13 - Organizing Information Efficiently

Big Idea

This lesson ties together the week's major theme: information should be organized so it can be found, updated, processed, and maintained efficiently.

Full Explanation

Collections, indexes, arrays, searching, and algorithms all connect through one central question: how should information be organized so it can be used efficiently? Good organization reduces work. Poor organization creates extra work.

Software often manages users, products, transactions, messages, images, documents, and locations. The better that information is organized, the easier it becomes to retrieve and process it.

Real-World Connection - Photography Files

Imagine storing every photo you have ever taken in one giant folder. After a few years, that folder could contain tens of thousands of images.

- Poor organization: every photo, receipt, video, and document mixed together.
- Better organization: folders by year, event, client, class, rider, or location.
- The files are the same. The structure makes them usable.

Connecting the Core Ideas

Concept	Question It Answers
Data structures	How should I organize information?
Indexes	How do I quickly locate something when I know its position?
Searching	How do I find something when I do not know its exact position?
Algorithms	What steps should I follow to solve the problem?

Code Example:

Python — Run — Submit code

```python
scores = [25, 52, 83]

def safely_get_first(scores):
    if len(scores) == 0:
        return 0

    first_score = scores[0]
    return first_score

print(safely_get_first(scores))
```

Output

```
25
```

Made with Goodnotes

Lesson 13 - Organizing Information Efficiently

Code Example - Concepts Working Together

```
students = ["Sophia", "CJ", "Caleb", "Emma", "Lucas"]
target = "Caleb"

for student in students:
    if student == target:
        print("Student Found")
```

What this combines: a collection, stored data, searching, and an algorithm. Week 1 concepts are not separate; they work together.

Why Organization Matters More as Data Grows

Collection Size	Organization Impact
Small: 10 items	Almost any organization can work.
Medium: 1,000 items	Good organization starts becoming important.
Large: 1,000,000 items	Good organization becomes critical.

Out-Loud Programmer Thinking

When building software, ask: What information am I storing? How will users need to access it? What structure makes that easiest? Those questions often matter more than the individual lines of code.

Terms and Definitions

Organization: Arranging information in a meaningful way.
Retrieval: Obtaining stored information.
Structure: The arrangement used to store information.
Scalability: The ability to continue performing well as data grows.
Maintainability: How easy a system is to understand and update.

Common Mistakes and Helpful Tips

- Mistake: Thinking organization is just housekeeping. Reality: organization affects performance and usability.
- Mistake: Focusing only on code. Reality: data organization often matters more than individual lines of code.
- Tip: Ask what problem the structure solves, what makes it useful, what tradeoffs it involves, and when you would choose it.

Notes

Made with Goodnotes

Code Example:

Python Run Submit code

```python
scores = [25, 50, 75, 100]

def safely_get_top_three(scores):
    if len(scores) < 3:
        return 0

    top_three_scores = scores[0], scores[1], scores[2]
    return top_three_scores

print(safely_get_top_three(scores))
```

Output

```
(25, 50, 75)
```

Python Run Submit code

```python
scores = [10, 20, 30, 40]

def safely_get_top_three(scores):

    if len(scores) == 0:
        return []

    elif len(scores) == 1:
        return [scores[0]]

    elif len(scores) == 2:
        top_two_scores = [scores[0], scores[1]]
        return top_two_scores

    else:
        top_three_scores = [scores[0], scores[1], scores[2]]
        return top_three_scores

print(safely_get_top_three(scores))
```

Output

```
[10, 20, 30]
```

Made with Goodnotes

Code Example:

Python

Run | Submit code

```python
scores = [5, 10, 15]

def safe_third_socre(scores):

    if len(scores) < 3:
        return 0
    return scores[2]

print(safe_third_socre(scores))
```

Output

```
15
```

Python

Run | Submit code

```python
scores = [100, 90, 80, 70]

def avg_top_three(scores):

    if len(scores) < 3:
        return None

    return (scores[0] + scores[1] + scores[2]) / 3

print(avg_top_three(scores))
```

Output

```
90.0
```

Made with Goodnotes

Lesson 14 - Week 1 Integration and Review

Big Idea

Week 1 introduced the foundation for CS102: information must be organized and processed efficiently. This lesson connects the whole week into one mental model.

Full Explanation

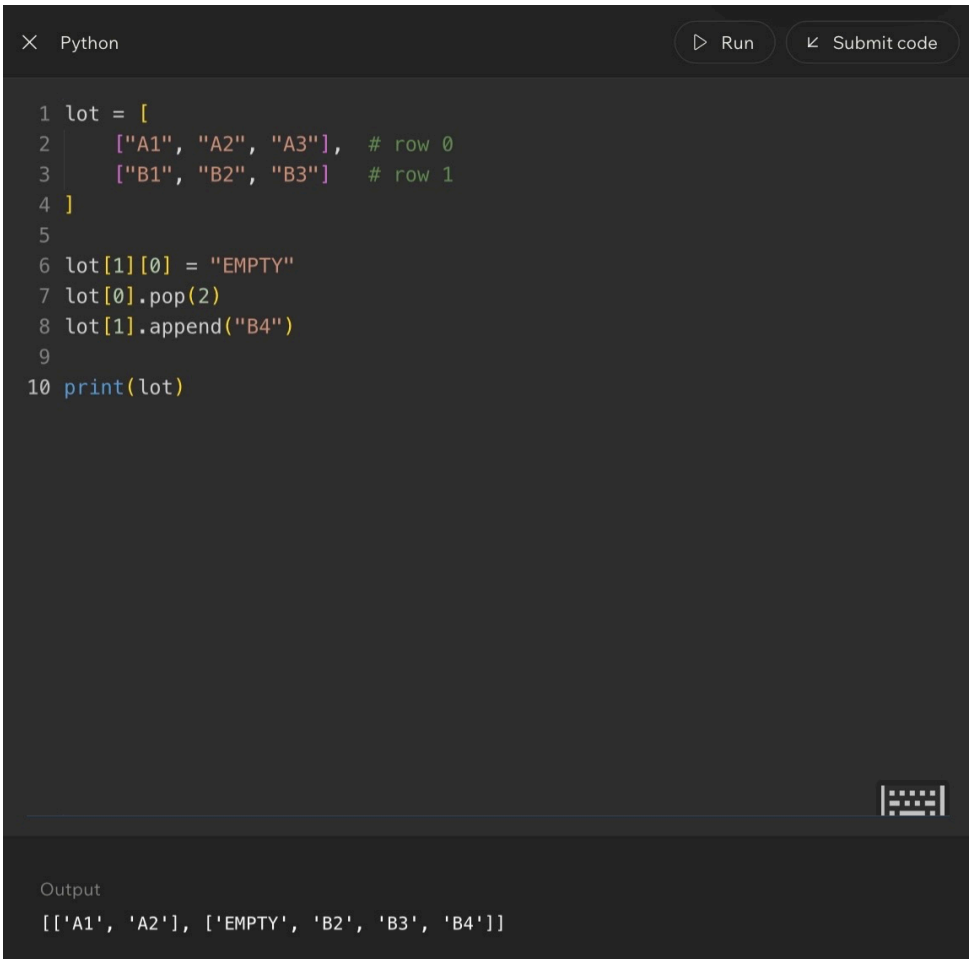
The week began with data structures and organization, then moved into indexes, arrays, access, updates, additions, removals, searching, and algorithms. The final pieces were efficiency and comparison: not every correct solution requires the same amount of work.

Together, these topics form the base for understanding more advanced data structures and algorithms later in the course.

Week 1 Concept Map

Flow	Meaning
Data Structures	Organize information.
Collections	Store related data together.
Indexes	Point to positions.
Arrays	Store values in ordered positions.
Access / Update / Modify	Use and change stored information.
Searching	Find information when location is unknown.
Algorithms	Follow repeatable steps.
Efficiency	Measure how much work is required.

Code Example:



Made with Goodnotes

Lesson 14 - Week 1 Integration and Review

How the Lessons Connect

Lesson	Main Focus
1	Course orientation and software engineering mindset
2	Data structures and organization
3	Indexed access
4	Arrays
5	Accessing elements
6	Updating values
7	Dynamic collections and tradeoffs
8	Adding and removing data
9	Searching
10	Algorithms
11	Linear Search
12	Comparing search strategies
13	Efficient organization
14	Integration and review

Real-World Example - Running a Business

- Store information: customers, products, orders, and employees.
- Organize information: databases, files, and records.
- Find information: customer accounts, inventory, and transactions.
- Process information: purchases, refunds, reports, and analytics.

Everything from Week 1 contributes to those tasks.

Out-Loud Programmer Thinking

Why do data structures exist? Because organization matters.
Why do algorithms exist? Because problems need repeatable solutions.
Why compare solutions? Because some solutions require less work than others.

Code Example:

Python ▷ Run ↙ Submit code

```
storage = [
    [   # floor 0
        ["F0B0-item1", "F0B0-item2"],   # box 0
        ["F0B1-item1"]                  # box 1
    ],
    [   # floor 1
        ["F1B0-item1", "F1B0-item2"]    # box 0
    ]
]

storage[0][0].append("F0B0-item3")
storage[1].append(["F1B1-item1"])
storage[0][1].pop(0)

print(storage)
```

Output

```
[[['F0B0-item1', 'F0B0-item2', 'F0B0-item3'], []], [['F1B0-item1', 'F1B0-item2'], ['F1B1-item1']]]
```

Made with Goodnotes

Code Example:

Python

Run | Submit code

```python
playlists = [
    [101, 102, 103],
    [],
    [201]
]

playlists[0].pop()
playlists[1].append(999)
playlists[2][0] = 300

print(playlists)
```

Output

```
[[101, 102], [999], [300]]
```

Python

Run | Submit code

```python
seats = [
    ["A1", "A2", "A3", "A4"],  # row 0
    ["B1", "B2", "B3", "B4"]   # row 1
]

seats[0].pop(1)
seats[1][2] = "TAKEN"
seats[0].insert(0, "VIP")
seats[1].append("B5")

print(seats)

#[[“VIP”, “A1”, “A3”, “A4”], [“B1”, “B2”, “TAKEN”, “B4”, “B5”]]
```

Output

```
[['VIP', 'A1', 'A3', 'A4'], ['B1', 'B2', 'TAKEN', 'B4', 'B5']]
```

Made with Goodnotes

Lesson 14 - Week 1 Integration and Review

Week 1 Master Summary

Concept	What to Remember
Data Structures	Organize information.
Indexes	Provide direct access.
Arrays	Store ordered collections.
Searching	Locates information.
Algorithms	Provide repeatable procedures.
Efficiency	Measures the work required.
Scalability	Describes how performance changes as data grows.

Common Week 1 Misconceptions

- Data structures are just lists. Reality: lists are only one example.
- Algorithms must be complicated. Reality: even simple instructions can be algorithms.
- If a solution works, it must be good. Reality: efficiency matters too.
- Indexes begin at 1. Reality: Python uses zero-based indexing.

Final Review Questions

1. What is a data structure?
2. What is an algorithm?
3. Why are indexes useful?
4. What is direct access?
5. How does Linear Search work?
6. What is efficiency?
7. Why does organization matter?
8. How do data structures and algorithms work together?
9. What is scalability?
10. What was the most important concept from Week 1?

Key Takeaways

- Data structures organize information.
- Algorithms process information.
- Organization improves efficiency.
- Searching locates data.
- Indexes provide direct access.
- Good software depends on thoughtful design decisions.
- Week 1 provides the foundation for the rest of CS102.

Notes

Made with Goodnotes

Expanded Glossary

How to Use This Glossary

Connect every term to three things: what it means, why it matters, and what beginner mistake it prevents.

Core Terms - Part 1

Term	Meaning	Example / Common Confusion
Algorithm	Step-by-step procedure used to solve a problem or complete a task.	Recipe, GPS directions, or a search procedure. Not always code.
Array	Ordered collection of elements stored in indexed positions.	Positions matter as much as values.
Collection	Group of related data stored together.	Students, scores, products, colors, photos.
Data	Information that can be stored, processed, and used.	Names, prices, dates, scores, temperatures.
Data Structure	Method of organizing information so it can be used efficiently.	The structure is the organization system, not the data itself.
Direct Access	Retrieving information immediately using its index.	colors[2] jumps directly to the value at position 2.
Efficiency	Amount of work required to complete a task.	Finding a song instantly vs. checking thousands manually.
Element	Individual value stored inside a collection.	Each color inside a colors list is an element.

Tiny Comparison

Concept	Plain-English Role
Data	The information.
Data structure	How the information is organized.
Algorithm	The process that works with the information.
Efficiency	How much work that process requires.

Glossary Notes

Made with Goodnotes

Expanded Glossary

Core Terms - Part 2

Term	Meaning	Example / Common Confusion
Index	Numerical position of an element inside a collection.	Indexes are positions, not values.
Indexed Access	Using an index to retrieve information directly.	students[0] retrieves the first stored student.
Linear Search	Search algorithm that checks elements one at a time until a match is found.	Works even when data is not organized.
Match	When the target value and current value are the same.	Searching for 'green' and finding 'green'.
Scalability	How well a system performs as data grows.	A solution for 10 items may struggle with 10 million.
Search	Process of locating information.	Finding a contact, product, message, or record.
Sequential Access	Checking information one item at a time.	Searching every locker until the correct one appears.
Target	The value being searched for.	target = 'green'
Zero-Based Indexing	Numbering system where positions begin at zero.	Humans count 1,2,3. Python positions start 0,1,2.

Quick Self-Test

Can you explain the difference between a value and an index? Can you explain why Linear Search still works on unsorted data? Can you explain why scalability matters before a program gets huge?

Memory Hook

Index = address. Value = information at that address. Linear Search = check one item at a time. Scalability = what happens when the data grows.

Glossary Notes

Made with Goodnotes

Concept Connections + Common Mistakes

Week 1 Concept Chain

Concept	How It Connects
Data	The raw information: names, scores, photos, messages.
Data structures	Organize the information so it can be used.
Collections	Hold groups of related data.
Indexes	Give positions so information can be accessed directly.
Arrays	Store ordered, indexed values.
Access + updates	Retrieve or change stored information.
Searching	Find information when the exact location is unknown.
Algorithms	Provide repeatable steps for solving the problem.
Efficiency + scalability	Measure work now and performance as data grows.

Common Mistakes by Topic

Topic	Mistake	Reality
Data structures	Thinking they are just vocabulary.	They are organization systems.
Indexes	Starting at 1.	Python starts at 0.
Arrays	Confusing values and positions.	Indexes are positions; elements are values.
Searching	Assuming the target exists.	Searches can fail.
Algorithms	Thinking they must be complicated.	Simple steps can be algorithms.
Efficiency	Ignoring work required.	Correct solutions can require very different work.

If You Remember Nothing Else

Data structures organize information. Indexes provide addresses. Arrays store ordered collections. Algorithms solve problems. Efficiency matters because working is not the same as working well.

Connection Notes

Made with Goodnotes

Review Question Breakdown + Exam Traps

Question Types and Answer Strategies

Question Type	Example	Strategy
Definition	What is a data structure?	Define it and explain its purpose.
Application	Why is a playlist a data structure?	Explain how information is organized.
Index	What does colors[1] return?	Label positions first, then answer.
Search	How does Linear Search work?	Explain step-by-step checking.
Comparison	Sequential vs indexed access	Focus on how retrieval happens.

Common Exam Traps and Fixes

Trap	What Usually Goes Wrong	How to Avoid It
Values vs indexes	colors[1] returns the second value, not the first.	Index = address. Value = data stored there.
Zero-based indexing	The third item is items[2], not items[3].	Write index labels above values.
Search failure	The target may not exist.	Always consider what happens if not found.
append vs insert	append adds to end; insert adds at a specific position.	Ask whether the new value belongs at the end or in the middle.
Correct vs efficient	Two answers can both work, but one may require much more work.	Ask: how many checks or steps?

Index Trap Example

```
colors = ["red", "blue", "green"]

Index:   0      1       2
Value: red   blue   green

colors[1] -> blue
```

Always label indexes before answering code-output questions.

Review Notes

Made with Goodnotes

Cheat Sheet + Exam-Day Quick Reference

Week 1 Cheat Sheet

Term	Fast Meaning	Example / Test Clue
Data Structures	Organize information	Playlist, contact list, array
Algorithms	Solve problems	Recipe, GPS directions, Linear Search
Arrays	Store ordered data	Index: 0 1 2 / Value: A B C
Elements	Individual values	'red' inside a colors list
Indexes	Positions	colors[0] is the first value
Direct Access	Jump directly to information	items[3]
Sequential Access	Check one item at a time	Linear Search
Efficiency	Amount of work required	How many checks?
Scalability	Performance as data grows	Will this still work with huge data?

Exam-Day Quick Reference

Term	Meaning
Data Structure	Organization system
Algorithm	Step-by-step procedure
Array	Ordered collection
Element	Stored value
Index	Position
Search	Find information
Linear Search	Check items one at a time
Efficiency	Work required
Scalability	Growth performance

Final Memory Hook

Data Structures = how information is organized.
Algorithms = how problems are solved.
Indexes = addresses.
Arrays = organized collections.
Searching = finding information.
Efficiency = work required.
Scalability = what happens when data grows.

Everything comes back to one question: How can information be organized and processed efficiently?

Quick Reference Notes

Made with Goodnotes

Practice Exercises + Visual Diagrams

Practice Set - Lessons 2 through 8

Lesson	Question	Answer / Check
Lesson 2	Which is a data structure: playlist, filing cabinet, contact list, or all of the above?	All of the above.
Lesson 3	letters = ['A','B','C','D']; what does letters[2] return?	C
Lesson 4	Label indexes for colors = ['red','blue','green','yellow'].	0 red, 1 blue, 2 green, 3 yellow
Lesson 5	nums = [10,20,30]; print(nums[1])	20
Lesson 5	colors.append('green') after ['red','blue']	['red', 'blue', 'green']
Lesson 6	scores = [90,85,80]; make the second score 95.	scores[1] = 95
Lessons 7-8	letters = ['A','B','D']; insert 'C' between B and D.	letters.insert(2, 'C')

Trace the Search

Collection	Target	Order of Comparisons
['red','blue','green','yellow']	yellow	red -> blue -> green -> yellow

Visual Diagram Collection

```
Sequential vs Indexed Access
Sequential: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7
Indexed: jump directly to 7

Array Visualization
Index:  0   1   2   3
Value:  A   B   C   D

Linear Search
Searching for F in A B C D E F
Path: A -> B -> C -> D -> E -> F
```

Use these diagrams when you need to explain a concept quickly in plain English.

Practice Notes

Made with Goodnotes

Comprehensive Review + Programmer Thinking

One-Sentence Review by Lesson

Lesson	Core Idea
1	CS is about organizing information and solving problems efficiently.
2	Data structures organize information.
3	Indexes provide direct access.
4	Arrays store ordered collections.
5	Indexes allow retrieval of stored values.
6	Values can be updated without changing positions.
7	Collections often need to grow and adapt.
8	Adding and removing data affects organization.
9	Searching locates information.
10	Algorithms provide repeatable solutions.
11	Linear Search checks one item at a time.
12	Correctness and efficiency are different.
13	Organization affects performance.
14	All concepts work together.

Beginner Thinking vs Programmer Thinking

Topic	Beginner Thinking	Programmer Thinking
Data Structures	Where do I store it?	How should I organize it so future tasks are easier?
Arrays	An array is a list.	An array is a collection of indexed positions.
Searching	Did I find it?	How much work did it take to find it?
Algorithms	What's the answer?	What process consistently produces the answer?
Efficiency	It works.	Will it still work well with a million records?

Week 1 Mindset Shift

Organizing information properly often matters more than writing clever code. A programmer thinks about structure, future operations, efficiency, and growth before getting attached to the first working solution.

Comprehensive Review Notes

Made with Goodnotes

Debugging Corner + Final Reflection

Debugging Corner

Issue	Example	Fix / Lesson
IndexError	colors = ['red','blue']; print(colors[5])	Position 5 does not exist. Check collection size and valid indexes.
Wrong index	Expected green, but colors[1] returns blue.	The programmer forgot zero-based indexing. Draw Index: 0 1 2 above values.
Wrong update	scores[0] = 95 changed the first score, not the second.	Label positions before replacing values.
Search failure	target = 'purple' is not in ['red','blue','green'].	Not every search succeeds. Always consider failure cases.

Debugging Checklist

1. Check indexes.
2. Check values.
3. Check collection size.
4. Trace the algorithm manually.
5. Ask what you expected, then compare that to what actually happened.

Week 1 Concept Map

WEEK 1 -> DATA -> DATA STRUCTURES -> COLLECTIONS -> INDEXES -> ARRAYS -> ACCESS / UPDATE / MODIFY -> SEARCHING -> ALGORITHMS -> EFFICIENCY -> SCALABILITY

Final Week 1 Reflection

Week 1 teaches that software development is fundamentally about organizing information and solving problems efficiently. Everything else in CS102 builds on that idea.

Final Notes

Made with Goodnotes